

Massively による GPU Smoothed Particle Hydrodynamics : セルグラフと入れ子の SegmentIterator による動的近傍の表現

Akira Hayakawa

2026 年 7 月 18 日

概要

GPU 上に Smoothed Particle Hydrodynamics (SPH) を実装する際の中心的な困難は、粒子対の式自体ではなく、移動し長さが一定でない近傍である。本稿では、この workload を Massively v0.80 の iterator algebra でどのように表現したかを説明する。一様格子セルの隣接関係を compressed sparse row (CSR) グラフとして一度だけ保存する。導関数を評価するたびに粒子状態をセル順に sort し、lower bound によりセルから粒子への第 2 の CSR 分割を作る。続いて、遅延 permutation と 2 層の **SegmentIterator** により、両者を query 粒子ごとの segment-of-segments へ合成する。**Segment** はコピーされた近傍データではなく境界を GPU 値として持つ read-only な意味的 view であるため、大域的な粒子対リストは実体化されない。1 回の transform invocation が 1 個の query 粒子を所有し、すべての寄与を private な accumulator へ gather するので、atomic 演算と write conflict を必要としない。また、Massively の型レベル上限である 13 個の read leaf を満たすため、solver をどのように分解したかを示す。end-to-end の物理的な確認には Sod shock tube を用い、 $t = 0.15$ における sample 密度の L_1 誤差は 0.004199 となった。65,536 粒子では、GPU 密度と CPU 全粒子対 reference の最大相対差は 5.89×10^{-6} であった。内蔵 Radeon 680M では、完全な 2 次元密度 snapshot が 5.109 ms であり、CPU cell-list 計算の 16.204 ms に対して高速であった。以上は新しい SPH 離散化の提案ではなく、SPH を型付き GPU iterator system へ写像する際の障害を特定し、Massively による対応する構成を示し、その利点と費用を測定する実装研究である。

1 はじめに

Smoothed Particle Hydrodynamics は、Euler 格子を移動粒子と compact support を持つ補間 kernel で置き換える [Gingold and Monaghan, 1977, Monaghan, 1992]。粒子 i について、密度、圧力、人工粘性は近傍粒子に関する和である。数値式は i について自然に並列化できるが、寄与する粒子集合は粒子が動くたびに変わる。そのため GPU 実装では、位置から不揃いな近傍関係を繰り返し構築し、同時に効率的な状態アクセスと有界な memory 使用量を維持しなければならない。

従来の GPU SPH 実装では、この問題を sort 済み cell list、linked cell、Verlet list、または専用の相互作用 kernel により解決している [H rault et al., 2010, Dom nguez et al., 2011, 2013, Winkler et al., 2018]。Massively は異なる programming 手段を提供する。CubeCL 上に、device algorithm と logical iterator の型付き algebra を提供する [Hayakawa, 2026, Tracel AI, 2026]。zip、transform、permutation、segmentation を合成し、実行を伴う algorithm がその式を GPU kernel へ lowering する。本研究の問いは次のように明確化できる。

粒子ごとに近傍リストを実体化せず、移動する SPH 近傍を Massively iterator としてどう表現するか。また、完全な solver に残る GPU 上の制約をどう処理するか。

本稿の貢献は以下である。

- 静的なセル CSR、動的に再構築するセル-粒子分割、遅延 permutation、および入れ子の segmented iteration を合成して SPH 近傍を得る方法を明示的に導出した。
- 各 GPU invocation が 1 個の query 粒子を評価して 1 個の出力を書く race-free な gather 形式を示し、その memory 量と計算量の trade-off を明らかにした。

- Massively v0.80 の read arity 上限 13 のもとで、1 次元・2 次元の密度、圧力、仕事、および粘性 kernel を field 単位でどう分解するかを示した。
- 構造テストと differential test、Sod shock tube、および密度 pipeline benchmark による検証結果を示した。

本実装は固定平滑化長、cubic spline、一次精度時間積分、および人工粘性を用いる。end-to-end の物理検証には標準的な Sod shock tube を用いる。

2 SPH が突きつける実装上の問題

2.1 粒子対 workload

粒子の位置 $\mathbf{x}_i$ 、速度 $\mathbf{v}_i$ 、質量 m_i 、密度 ρ_i 、比内部エネルギー e_i 、および平滑化長 h について、実装した密度推定は

$$\rho_i = \sum_j m_j W_d(\|\mathbf{x}_i - \mathbf{x}_j\|, h) \quad (1)$$

である。理想気体の状態方程式は $p_i = (\gamma - 1)\rho_i e_i$ である。圧力と粘性の導関数は一般に次の SPH 形式を持つ。

$$\frac{d\mathbf{v}_i}{dt} = - \sum_j m_j \left(\frac{p_i}{\rho_i^2} + \frac{p_j}{\rho_j^2} + \Pi_{ij} \right) \nabla_i W_{ij}, \quad (2)$$

$$\frac{de_i}{dt} = \sum_j m_j \frac{p_i}{\rho_i^2} (\mathbf{v}_i - \mathbf{v}_j) \cdot \nabla_i W_{ij} + \frac{1}{2} \sum_j m_j \Pi_{ij} (\mathbf{v}_i - \mathbf{v}_j) \cdot \nabla_i W_{ij}. \quad (3)$$

これらは標準的な保存型 SPH の形式である [Monaghan, 2005, Price, 2012]。実装では cubic B-spline

$$W_d(r, h) = \frac{\sigma_d}{h^d} \begin{cases} 1 - \frac{3}{2}q^2 + \frac{3}{4}q^3, & 0 \leq q < 1, \\ \frac{1}{4}(2 - q)^3, & 1 \leq q < 2, \\ 0, & q \geq 2, \end{cases} \quad q = r/h \quad (4)$$

を用いる。ここで $\sigma_1 = 2/3$ 、 $\sigma_2 = 10/(7\pi)$ である。したがって、粒子対が寄与するのは $r < 2h$ の場合だけである。

人工粘性は接近する粒子についてのみ評価する。無次元座標 $\mathbf{q}_{ij} = (\mathbf{x}_i - \mathbf{x}_j)/h$ を用いると、実装は係数形式

$$\mu_{ij} = \frac{(\mathbf{v}_i - \mathbf{v}_j) \cdot \mathbf{q}_{ij}}{\|\mathbf{q}_{ij}\|^2 + 0.01}, \quad \Pi_{ij} = \frac{-A\mu_{ij} + B\mu_{ij}^2}{\hat{\rho}_{ij}} \quad (5)$$

を持つ。1 次元では係数を設定可能とし、密度の算術平均を用いる。現在の 2 次元 workload では $A = 1$ 、 $B = 2$ 、 $\hat{\rho}_{ij} = \sqrt{\rho_i \rho_j}$ である。この数値的な選択自体は本稿の貢献ではない。ただし、その因数分解は Massively の read arity 制約内で演算を表現するため、後に重要となる。

式 (1)–(3) から GPU workload の特徴が分かる。出力粒子ごとに候補近傍数が異なり、各候補について query 側と neighbor 側の複数 column を必要とする。また同じ空間関係を、異なる column tuple を用いて密度、圧力、粘性で再利用する。

2.2 同時に満たすべき 5 つの条件

実装は次の 5 条件を同時に満たす必要がある。

1. **完全で動的な近傍**。任意に粒子が移動した後も、support 内の全粒子対を $O(N^2)$ scan なしで訪問しなければならない。
2. **整合した並べ替え**。位置だけを sort し、速度、質量、energy、粒子 identity を sort しなければ、異なる粒子の状態を気付かずに混合してしまう。
3. **race-free な集約**。対称な粒子対演算で i と j の両方を更新すると、GPU 上で atomic 演算または coloring が必要になる。

4. **有界な表現**。全候補粒子対の実体化は、粒子ごとの候補数の総和に比例する memory を追加し、粒子移動後に再構築しなければならない。
5. **表現可能な kernel**。合成した入力、Massively v0.80 が扱える静的 iterator 型と kernel read arity に収まらなければならない。

粒子データを device 上に保持することも実用上の条件である。そうしなければ、正しい近傍 kernel を実装しても、各 time step の転送時間が支配的になり得る。

3 solver で使用する Massively の機構

3.1 logical iterator と実行点

Massively は logical な入力式と、allocation および実行を伴う algorithm を区別する。device vector の slice は iterator であり、zip は column を結合し、`lazy::transform` は index から読む値を変換し、`lazy::permute(values, indices)` は `values[indices[i]]` を読む。これらの式は、それ自体では結果 array を作らない。`vector::transform`、`vector::sort_by_key`、`vector::lower_bound` のような operation が、device storage を生成する実行点である。

この区別が設計の中心となる。solver は、大域的に変化する必要がある data、すなわち sort 済み粒子 column、cell ID、cell offset、物理量の出力を実体化する。一方、それら array 間の関係は遅延 reader として表現する。

3.2 SegmentIterator が表すもの

logical な値 stream V と offset $P = (P_0, \dots, P_K)$ について、Massively は

$$\text{Seg}(V, P)[k] = V[P_k : P_{k+1}] \quad (6)$$

を定義する。この `SegmentIterator` の item 型は `Segment<V::Item>` である。`Segment` は所有権を持つ vector ではなく、allocation 機能も write 機能も持たない。生成される GPU 値では、register に保持される `start` と `len` の境界だけが segment ごとに異なる。backing reader は、その式の全 segment が共有する compiler 側の context である。CubeCL operation の内部では、`len()` が動的な長さを返し、`at(k)` が backing expression の `start + k` からの read を生成する。

SPH にとって重要な帰結が 2 つある。第 1 に、粒子 data をコピーせず segment を permute できる。第 2 に、ある segment iterator の値自体を segment にでき、外側の `Segment` の item を内側の `Segment<ParticleFields>` にできる。これは「近傍セルと、それぞれの近傍セル内の粒子」を表すために必要な型そのものである。

3.3 read arity の境界

Massively は canonical な入力式に型レベルの read arity を割り当てる。v0.80 で利用可能な arity は $A_1, \dots, A_{13}$ であり、型レベルの加算は合計が 13 以下の場合にのみ定義される。この仕組みにより過大な合成を compile time で検出できるが、物理的には自然な単一の SPH 式を表現できない場合もある。Section 5 では、同じ近傍 view を保ちながら実装をどう分割するかを示す。

4 移動する近傍の構築

4.1 静的セルグラフ

cell 幅を compact support 半径と等しい $s = 2h$ とする。1 次元では各 cell は自分自身と最大 2 つの隣接 cell に接続する。2 次元では、domain 境界で切り取られた 3×3 block に接続する。この関係は domain の幾何形状に依

存するが粒子位置には依存しないため、一度だけ CSR として構築する。

$$A = (A_0, \dots, A_{E-1}) \quad \text{近傍 cell ID,} \quad (7)$$

$$G = (G_0, \dots, G_C) \quad \text{row offset,} \quad (8)$$

$$\mathcal{G}(c) = A[G_c : G_{c+1}]. \quad (9)$$

C 個の cell に対し、1 次元では $E \leq 3C$ 、2 次元では $E \leq 9C$ である。

この graph は kernel support に対して完全である。 $\|\mathbf{x}_i - \mathbf{x}_j\| < 2h = s$ なら、各座標方向の絶対変位も s 未満である。したがって整数 cell 座標の差は各次元で高々 1 であり、CSR row は該当する全 cell offset を列挙する。2 次元の 3×3 block の角にあり、円形 support の外にある候補は、粒子対 loop 内の式 (4) により除外する。

4.2 動的なセル-粒子分割

導関数を評価するたびに、Massively の `vector::transform` で各位置を clamp 済み row-major cell ID c_i へ写像する。その後、structure-of-arrays の完全な状態を zip した値に対して、1 回の `sort_by_key` を実行する。tuple の field 数は 1 次元で 8、2 次元で 10 である。tuple 全体を一度に sort することで、次の不変条件を保つ。

$$(\text{id}, \mathbf{x}, \mathbf{v}, e, m, \text{事前計算した質量因子})_k \quad \text{はすべて同じ粒子に属する。} \quad (10)$$

sort 済み cell key も、並べ替えた状態とともに保持する。

全 cell ID $0, \dots, C$ について、`vector::lower_bound` で sort 済み key を検索する。その出力 $P = (P_0, \dots, P_C)$ が粒子 stream を分割する。

$$\mathcal{B}(c) = V[P_c : P_{c+1}] \quad (11)$$

空 cell に特別な処理は不要であり、単に $P_c = P_{c+1}$ となる。

4.3 入れ子 segment への合成

中心となる構成を Rust 型から独立に記述する。 V を、粒子の sort 順に並んだ任意の neighbor field の zip stream とする。次を定義する。

$$B = \text{Seg}(V, P), \quad B[c] = \mathcal{B}(c), \quad (12)$$

$$R = \text{Permute}(B, A), \quad R[e] = B[A_e], \quad (13)$$

$$H = \text{Seg}(R, G), \quad H[c] = (B[A_{G_c}], \dots, B[A_{G_{c+1}-1}]), \quad (14)$$

$$N = \text{Permute}(H, c), \quad N[i] = H[c_i]. \quad (15)$$

したがって $N[i]$ は query 粒子 i について、cell segment を要素とする segment である。型変換を省略した Massively の実際の pattern を Listing 1 に示す。

Listing 1 1 個の SPH 近傍 view を構築する Massively コードの概略。

```
let b = SegmentIterator::new(values, particle_offsets);
let r = lazy::permute(b, neighbor_cells);
let h = SegmentIterator::new(r, graph_offsets);
let n = lazy::permute(h, cell_ids);
let output = vector::transform(exec, zip(query, n), PairReduction)?;
```

第 1 の `SegmentIterator` は、実体化された sort 済み column と particle offset を backing data とする。 A による permutation、 G による第 2 の segmentation、 c_i による最後の permutation は、logical な read expression のままである。特に、全 (i, j) 粒子対を格納する array も、各 i の近傍 vector をコピーした array も存在しない。入れ子 view に必要な物理的 topology 入力 P, A, G, c だけである。

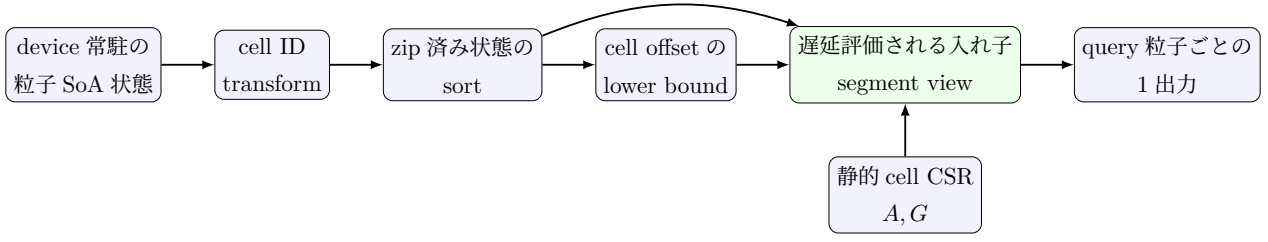


図 1 青は実体化される stage、緑は遅延 view である。粒子所属を再構築しても、静的 cell graph は再構築しない。

4.4 race-free な gather と cost

最後の `vector::transform` は query 粒子ごとに 1 個の logical operation を起動する。その operation は外側の cell segment と内側の particle segment を走査し、 $q \geq 2$ を除外し、operation local な CubeCL `RuntimeCell` へ集約する。invocation i は出力 i だけを書く。他の粒子の導関数を書く invocation はないため、atomic addition も粒子対 coloring も必要ない。

意図した trade-off として、相互作用する unordered pair は通常 2 回評価される。すなわち $i \rightarrow j$ と $j \rightarrow i$ である。scatter 形式なら粒子対の算術演算を再利用できるが、同期問題が再び生じる。ここでは、単純な ownership rule と連続した query 出力を優先して gather 形式を採用する。

Q を cell row が列挙する ordered candidate interaction の個数とする。保存する表現は $O(N + C + E)$ であり、永続的な候補 list の $O(N + Q)$ とは異なる。1 回の prepare は

$$T_{\text{prepare}} = T_{\text{sort}}(N) + O(N) + O(C \log N) + O(Q) \quad (16)$$

の work model を持つ。 T_{sort} は backend 依存のままとし、 $C + 1$ 個の lower-bound query が $O(C \log N)$ 項に対応する。cell 占有数が有界なら $Q = O(N)$ である。この解析は、不揃いな segment 長による divergence を消去するものではない。Massively は表現と lowering の仕組みを提供するが、実行する work 量は物理的な粒子分布に依存する。

5 完全な solver への写像

5.1 device 常駐 pipeline

粒子 identity、位置、速度、比内部エネルギー、質量、および m/h の冪を structure-of-arrays 形式の device vector へ upload する。1 step では次を実行する。

1. cell ID を計算し、完全な zip 済み状態を sort し、particle offset を得る。
2. 位置を h^{-1} で scale し、入れ子近傍を通して密度を評価する。
3. 理想気体の状態方程式を elementwise に評価する。
4. 同じ sort 済み粒子に対して圧力、圧力仕事、粘性用の型付き view を再構築し、それぞれの粒子対 reduction を評価する。
5. 導関数を合成し、energy、速度、位置を更新する。
6. 次 step のため新しい array を device に保持する。

明示的な device-to-host 転送を行うのは snapshot だけである。sort により storage 順が変わるため、snapshot は安定した粒子 ID を含む。必要であれば、device 外で物理出力を identity 順へ戻せる。

各物理 operator が必要とする neighbor tuple は異なるため、同じ空間近傍であっても型付き view を operator ごとに再構築する。密度は位置と質量因子を読み、圧力はさらに p/ρ^2 を読み、粘性は速度と密度因子を読む。これらの view は、実体化済みの P 、 A 、 G 、 c を共有する。

5.2 13 個の read leaf に収める方法

Table 1 は、合成された各入力 of 物理的 leaf 数を示す。4 個の topology leaf は、particle offset P 、neighbor cell ID A 、cell graph row offset G 、query cell ID c である。残りが query column と neighbor column である。

表 1 Massively v0.80 における read arity の分解。

operation	query field	neighbor field	topology	合計
1 次元密度	1	2	4	7
2 次元密度	2	3	4	9
1 次元圧力導関数	3	4	4	11
1 次元粘性導関数	5	4	4	13
2 次元圧力加速度	3	4	4	11
2 次元圧力仕事 (各 axis)	4	4	4	12
2 次元粘性導関数	4	5	4	13

1 次元粘性の式は上限に正確に到達する。2 次元では圧力加速度を 1 pass で評価し、圧力仕事を x と y の pass に分ける。両速度成分と全圧力 data を 1 個の近傍式へ zip すると、利用可能な型レベル arity を超えるためである。

2 次元粘性には、位置、両速度成分、質量 gradient 因子、密度が必要である。実装はまず

$$f_j = \frac{m_j}{h^3 \sqrt{\rho_j}} \quad (17)$$

を実体化し、13 leaf の粒子対 pass で f_j を 1 個の neighbor field として用いる。その後、得られた 3 個の導関数に elementwise な post-pass で $1/\sqrt{\rho_i}$ を掛ける。これにより式 (5) の幾何平均を実現する。この分割には 2 回の追加 dispatch と中間 array が必要であるが、一般的な Massively の手法を示している。すなわち、合成式が広すぎる場合、分離可能な粒子ごとの因子を不揃いな粒子対 reduction の外へ移動する。

5.3 課題と解決機構の対応

表 2 SPH 実装上の条件と Massively による実現の対応。

SPH/GPU 上の問題	Massively による実現	残る cost または trade-off
移動し不揃いな近傍	Cell-ID transform、zip 済み sort、lower bound、入れ子の <code>SegmentIterator</code> view	導関数評価ごとに sort と offset を再構築
候補 list の memory 量	遅延 permutation により、粒子対 data をコピーせず cell CSR と particle CSR を合成	候補の算術演算は走査時に実行
並列 write conflict	1 個の transform invocation が private state へ 1 query 粒子を gather	対称な unordered pair を通常 2 回評価
SoA column の整合性	1 個の tuple を <code>sort_by_key</code> の値とし、全 column を同時に移動	全 persistent field を tuple へ含める必要がある
幅の広い物理 operator	axis ごとの pass と分離可能な pre/post transform により A_{13} を満たす	追加 dispatch と一時 device vector
繰り返す host 転送	実行 algorithm は device 常駐 vector を生成し、snapshot のみ明示的に転送	host からの検査は同期点となる

6 検証と評価

6.1 実装レベルの check

多数の流体力学 benchmark を並べるのではなく、写像の failure mode に対応するようテストを層別化した。決定的 test と proptest による生成 test で、cubic spline の compact support と対称性、CSR row の一意性と対称性、境界・内部 row の期待次数、および多様な grid dimension における近傍完全性条件を検査する。続く GPU differential test では、1 次元・2 次元の入れ子 segment 密度を、式 (1) の直接的な CPU 全粒子対評価と比較する。完全な 1 step では全質量を保存し、境界との相互作用前には浮動小数点 tolerance 内で運動量を保存し、全状態が有限かつ正であることを要求する。

これらは、見た目には妥当な shock plot が見逃し得る error を分離する。例えば、斜め cell の欠落、CSR edge の重複、独立に sort された column、空 cell offset の誤りである。以上により近傍実装を検証し、次の shock tube を本稿唯一の end-to-end 物理検証とする。

6.2 Sod shock tube

Sod の初期状態は

$$(\rho, u, p)_L = (1, 0, 1), \quad (\rho, u, p)_R = (0.125, 0, 0.1), \quad \gamma = 1.4 \quad (18)$$

であり、不連続面を $x = 0.5$ に置く [Sod, 1978]。計算には 1,080 粒子、 $h/\Delta x_R = 1.2$ 、sample 区間外の変数状態 padding を用い、296 step で $t = 0.15$ まで進める。SPH 密度を等間隔の 401 点へ補間し、厳密 Riemann 解と比較する。

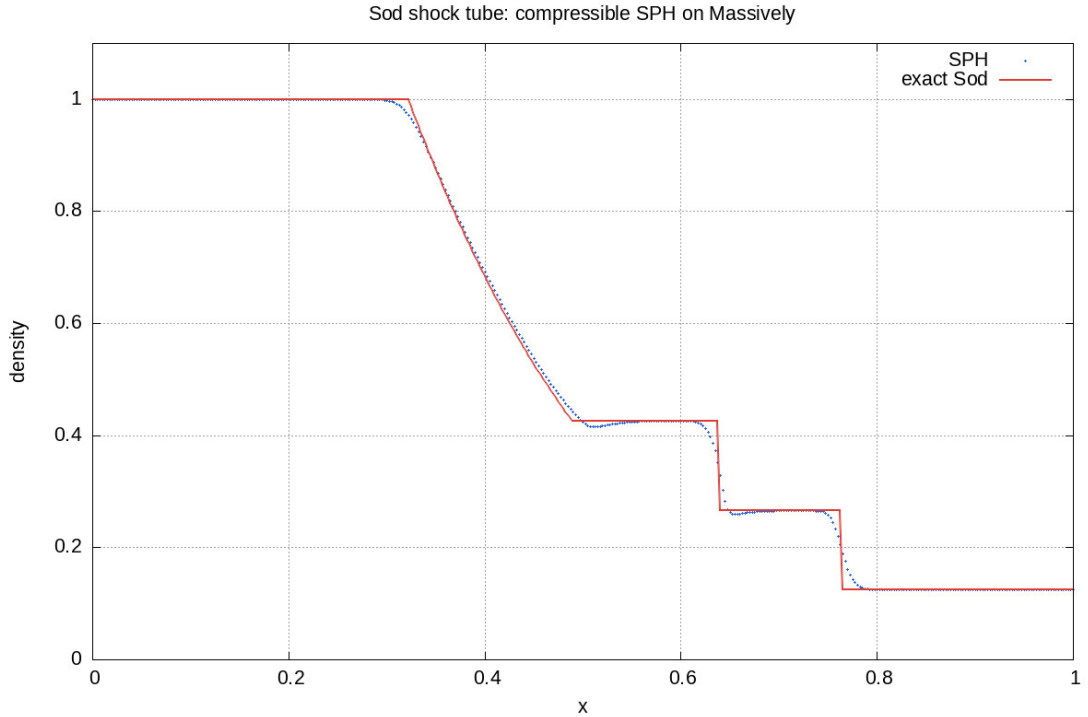


図2 $t = 0.15$ における Sod shock-tube 密度。GPU SPH 解は rarefaction、contact discontinuity、shock を再現している。

Figure 2 の sample 密度の平均絶対誤差は 0.00419917 である。計算された最小密度は 0.124903、最小圧力は 0.100179 であり、全粒子質量は変化しない。plot で使用する厳密 star state は $p_* = 0.303130$ 、 $u_* = 0.927453$

である。この結果は、1 個の発展問題において sort、密度、状態方程式、圧力・粘性導関数、時間積分、および snapshot 再構成の合成を確認する。これは収束性研究ではなく、あらゆる SPH flow regime を単独で検証するものでもない。

6.3 密度 pipeline の性能

benchmark は、内蔵 Radeon 680M を持つ AMD Ryzen 7 7735U、WGPU backend、Linux 6.17、Rust 1.96.0 で実行した。GPU 値と CPU cell-list 値は warm-up 後の 5 回の計測の中央値である。GPU snapshot には cell-ID 評価、sort、lower-bound 構築、入れ子密度、状態方程式、および snapshot の host 転送を含む。CPU cell-list の範囲は list 構築と密度のみである。したがって、両者の時間は近傍設計の engineering 上の証拠として有用だが、state-of-the-art 実装や完全な solver step の比較ではない。

表 3 2次元密度 benchmark (milliseconds)。

N	GPU snapshot	CPU cells	CPU all pairs	CPU/GPU	最大相対差
1,024	1.976	0.247	2.876	0.125	3.12×10^{-7}
4,096	2.018	0.984	44.764	0.488	1.61×10^{-6}
16,384	2.457	4.060	711.865	1.653	2.38×10^{-6}
65,536	5.109	16.204	—	3.171	5.89×10^{-6}

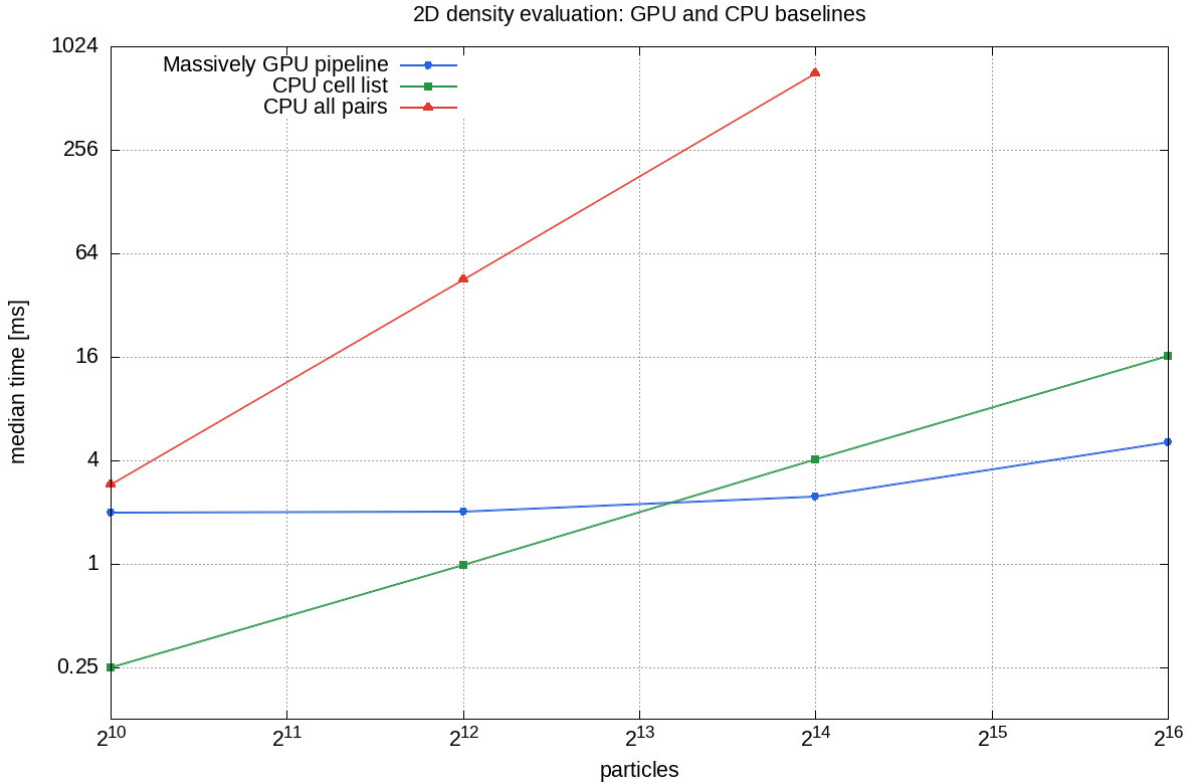


図 3 密度 pipeline の scaling。小規模入力では dispatch overhead が支配的であり、測定した machine では 4,096 粒子と 16,384 粒子の間で GPU が CPU cell 実装を上回る。

1,024 粒子では約 2 ms の GPU 起動・同期 cost により host 実装が速い。65,536 粒子では、GPU snapshot が状態方程式と host 転送を含むにもかかわらず、CPU cell-list の測定範囲より 3.17 倍速い。16,384 粒子では、CPU 全粒子対密度は GPU snapshot より約 290 倍遅い。正しさについてさらに重要なのは、全粒子対密度に対する最

大相対差が最大規模でも 5.89×10^{-6} に留まることである。この結果は、十分な粒子対 work があれば、入れ子 view が有用な GPU pipeline へ lowering されることを示す。ただし、sort、粒子対走査、転送の各寄与を分離した結果ではない。

6.4 再現性

Cargo lockfile は Massively tag v0.80 の revision f1041b5cf7902b01c303b2af3f69548e8c05cc91 を記録する。Rust workspace のテストは `cargo test --workspace` で実行する。plot script と生成した JPEG を `workspace/draw` と `workspace/plot` で version 管理する。英語論文と等価な日本語論文は、宣言した Docker 環境内で `just --justfile paper/justfile` により build し、`paper/paper.pdf` と `paper/paper-ja.pdf` を生成する。

7 議論と限界

本研究の中心的な結果は表現方法である。SPH は、各粒子から近傍粒子への可変 graph を必要とするように見える。本実装が保存するのは、cell から近傍 cell への静的 CSR と、cell から粒子への動的な sort 済み分割という 2 個の小さな関係だけである。入れ子 segmentation と permutation が、read 時に両者の関係合成を行う。これが `SegmentIterator` をこの問題に適合させる理由である。近傍関係を dense であるかのように扱わず、別の粒子対 array に変換することもなく、型付き iterator 式を通して不揃いな境界を運ぶことができる。

Massively を用いても、基礎的な cost が消えるわけではない。現在の solver は導関数評価ごとに sort する。粒子分布が非常に不均一なら load imbalance が残る。gather 走査は対称な粒子対 work を重複させる。13-read の境界により一部の 2 次元 operator は複数 dispatch へ分かれる。これらは隠された backend 詳細ではなく、明示的な設計上の trade-off である。

平滑化長は大域のかつ固定であり、時間積分は一次精度 symplectic Euler、境界は矩形反射、状態方程式は理想気体である。benchmark は 1 個の内蔵 GPU で密度 snapshot を測定したものであり、複数 device 上の完全な長時間 simulation ではない。Sod の結果は信頼できる end-to-end baseline を確立するが、より広い数値的主張には resolution sweep と追加の物理問題が必要である。

実装に直接関係する次の課題は以下である。

- 変位上限から cell 所属が不変と証明できる substep では、sort 順を再利用する。
- 走査 work の削減が追加の $O(Q)$ storage に勝る場合、Verlet 候補を option として cache する。
- sort、lower bound、入れ子走査、転送を個別に profile する。
- vector field の packing または Massively の read arity 拡張により、2 次元 pass 数を減らす。
- 同じ CSR/segment 構成を 3 次元へ拡張する。各 row は最大 27 cell となるため、その cell 占有数と divergence を評価する。

adaptive smoothing length も重要な数値的拡張である。ただし、cell 幅と粒子ごとの support を現在の固定 9-cell stencil で表せなくなるため、保守的な support rule を含む新たな実装問題でもある。

8 結論

移動する SPH 近傍は、粒子対リストを実体化せず Massively v0.80 で表現できる。本実装は静的 cell CSR を保存し、全粒子 column を整合して sort し、lower bound で cell-particle offset を導出し、遅延 permutation と 2 層の `SegmentIterator` により両関係を合成する。query ごとの gather transform が atomic 演算を不要にし、分離可能な pre/post pass が完全な solver を Massively の 13-read 型境界内に収める。Sod shock tube が発展 solver を確認し、differential density が近傍を全粒子対計算と照合し、測定された crossover が有用な GPU throughput を示す。同時に、繰り返す sort、対称 work の重複、load imbalance、追加 pass という cost を明示した。これに

より、次の Massively および SPH 実装を検討するための具体的な基盤が得られた。

参考文献

- José M. Domínguez, Alejandro J. C. Crespo, Moncho Gómez-Gesteira, and Jean-Christophe Marongiu. Neighbour lists in smoothed particle hydrodynamics. *International Journal for Numerical Methods in Fluids*, 67(12):2026–2042, 2011. doi: 10.1002/fld.2481.
- José M. Domínguez, Alejandro J. C. Crespo, and Moncho Gómez-Gesteira. Optimization strategies for CPU and GPU implementations of a smoothed particle hydrodynamics method. *Computer Physics Communications*, 184(3): 617–627, 2013. doi: 10.1016/j.cpc.2012.10.015.
- Robert A. Gingold and Joseph J. Monaghan. Smoothed particle hydrodynamics: Theory and application to non-spherical stars. *Monthly Notices of the Royal Astronomical Society*, 181(3):375–389, 1977. doi: 10.1093/mnras/181.3.375.
- Akira Hayakawa. Massively: Multi-platform GPU parallel algorithms for rust. Software, version 0.80, 2026. URL <https://github.com/akiradeveloper/massively>. Git commit f1041b5cf7902b01c303b2af3f69548e8c05cc91.
- Alexis Hérault, Giuseppe Bilotta, and Robert A. Dalrymple. SPH on GPU with CUDA. *Journal of Hydraulic Research*, 48(sup1):74–79, 2010. doi: 10.1080/00221686.2010.9641247.
- Joseph J. Monaghan. Smoothed particle hydrodynamics. *Annual Review of Astronomy and Astrophysics*, 30:543–574, 1992. doi: 10.1146/annurev.aa.30.090192.002551.
- Joseph J. Monaghan. Smoothed particle hydrodynamics. *Reports on Progress in Physics*, 68(8):1703–1759, 2005. doi: 10.1088/0034-4885/68/8/R01.
- Daniel J. Price. Smoothed particle hydrodynamics and magnetohydrodynamics. *Journal of Computational Physics*, 231(3):759–794, 2012. doi: 10.1016/j.jcp.2010.12.011.
- Gary A. Sod. A survey of several finite difference methods for systems of nonlinear hyperbolic conservation laws. *Journal of Computational Physics*, 27(1):1–31, 1978. doi: 10.1016/0021-9991(78)90023-2.
- Tracel AI. CubeCL: Multi-platform high-performance computing language extension for rust. Software, 2026. URL <https://github.com/tracel-ai/cubecl>.
- Daniel Winkler, Massoud Rezavand, and Wolfgang Rauch. Neighbour lists for smoothed particle hydrodynamics on GPUs. *Computer Physics Communications*, 225:140–148, 2018. doi: 10.1016/j.cpc.2017.12.014.